

Fondamenti di Analisi dei Dati

from **data analysis** to **predictive techniques**

Prof. Antonino Furnari (antonino.furnari@unict.it)

Corso di Studi in Informatica

Dip. di Matematica e Informatica

Università di Catania



Università
di Catania

Introduction to Predictive Analysis

Models, the two goals they serve, and how to tell whether they work



From Description to Prediction

Traditional Analysis

So far, we've focused on **understanding data** through:

- Summarizing with mean, median, variance
- Studying relationships via covariance and correlation
- Statistical inference to generalize findings

This approach is **passive** — we observe, compute scores, and perform tests.

Predictive Modeling

Now we shift to an **active approach** with two main goals:

- **Inference:** Understanding relationships between variables
- **Prediction:** Building models for unseen data

This field overlaps heavily with **machine learning** and requires creating formal predictive models.



Case Study: the CDC BRFSS Survey

Our examples use the **CDC Behavioral Risk Factor Surveillance System**, the annual health survey of the US Centers for Disease Control. The 2015 responses give **253 680 respondents** and **21 variables**, with a binary outcome: whether the respondent has ever been told they have diabetes.

BMI

Body mass index, the one continuous variable we will model directly

HighBP, HighChol, Stroke, HeartDiseaseorAttack

Conditions the respondent reports having been diagnosed with

Smoker, PhysActivity, Fruits, Veggies, HvyAlcoholConsump

Behaviour, all binary

GenHlth, MentHlth, PhysHlth, DiffWalk

Self-reported health, on a scale and in days

Age, Sex, Education, Income

Demographics, banded

Diabetes_binary

The outcome: 1 if the respondent has been told they have diabetes

- **13.9%** of respondents answer yes to the outcome. Keep that number in mind: it comes back when we score a model.





What Is a Model?

In data analysis, a **model is a simplified representation of reality**. It's not reality itself, but a formal abstraction designed to serve a specific purpose.

Consider a geographical map: it's flat whilst the Earth is spherical, and it omits countless details like trees, rocks, and buildings. **Yet this simplification makes it useful for navigation.**

"All models are wrong, but some are useful." — George Box

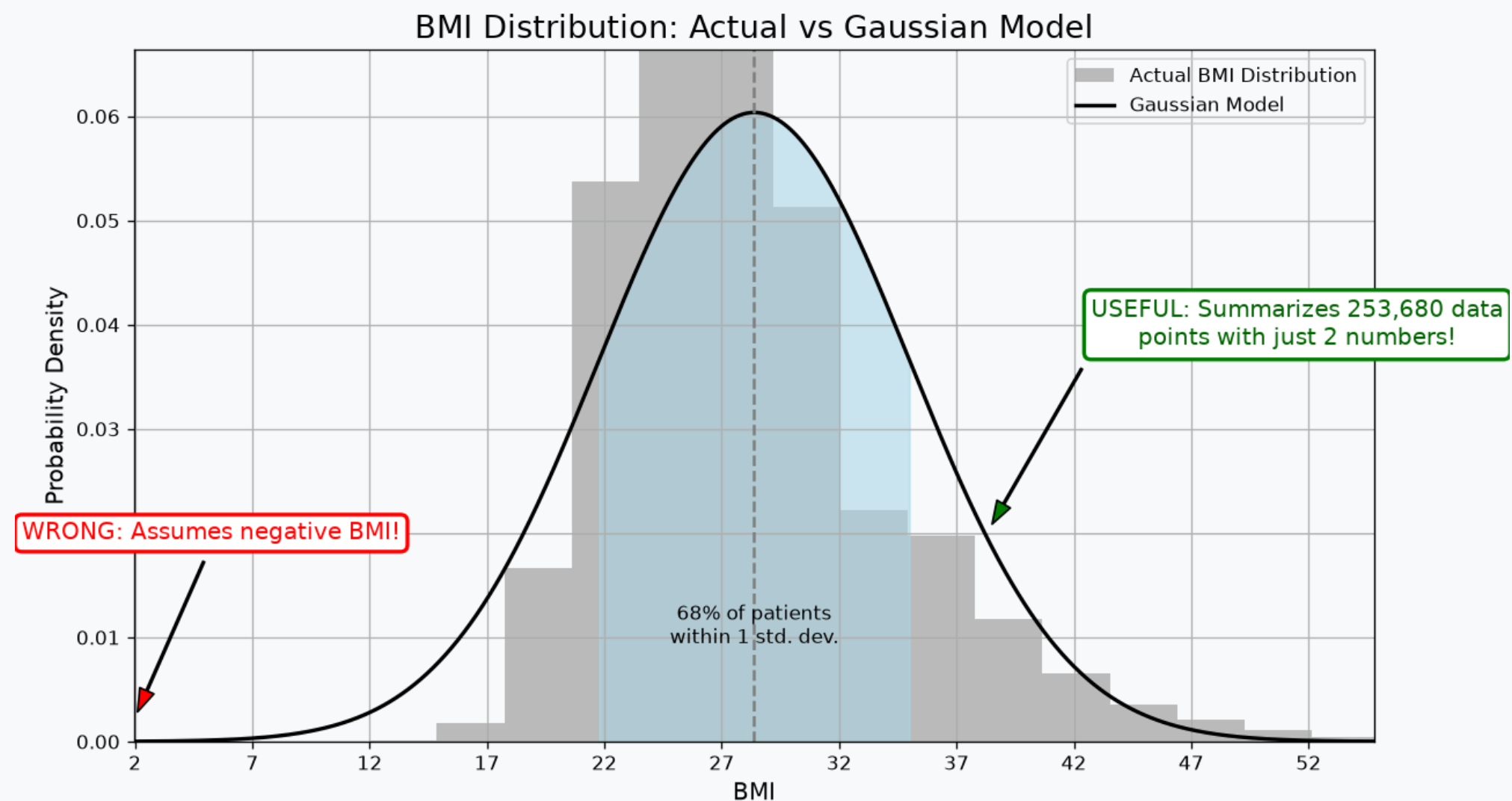
This famous aphorism captures the essence of modelling. A data model — whether statistical or machine learning — cannot capture every nuance of a complex real-world process.

But it becomes valuable if it helps us achieve our goal, whether understanding relationships or predicting outcomes.



Example: Modelling the BMI Distribution

Consider modelling the distribution of **Body Mass Index** in the survey using a **Gaussian** distribution. This model is defined by just two parameters: mean (μ) and standard deviation (σ). Whilst it is technically "wrong" — it assumes negative BMI values are possible and extends to infinity — it proves remarkably useful.

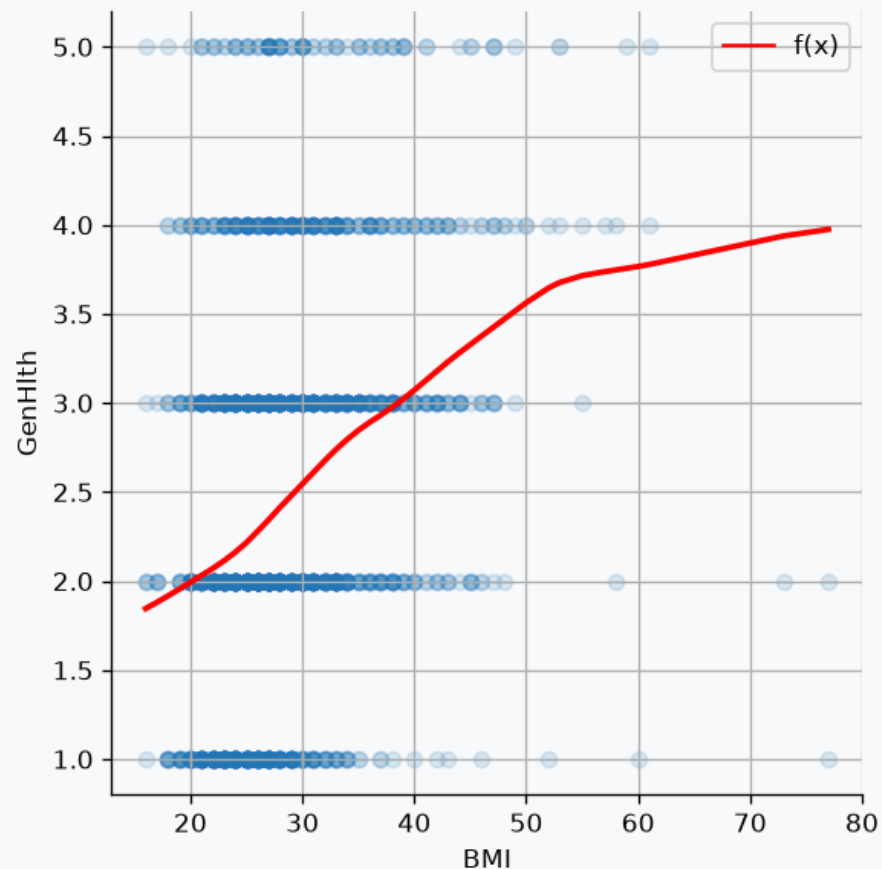


Reality check from the data:

Mean BMI: 28.38
Standard Deviation: 6.61
Patients within 1 std dev: 196003 out of 253680
Percentage within 1 std dev: 77.26%

Two numbers stand in for a quarter of a million. The fit is not perfect — the real distribution has a longer right tail than a Gaussian, and 77% rather than 68% of respondents fall within one standard deviation — but it tells us where the typical respondent is and how much spread there is around them.

The General Predictive Model Framework



When performing predictive analysis, we work with a dependent variable Y (what we want to predict) and a set of predictor variables $X = (X_1, X_2, \dots, X_n)$.

We express a predictive model in this general form:

$$Y = f(X) + \epsilon$$

Model Components

- $f(X)$: The predictive function we aim to discover through learning
- ϵ (epsilon): The error term capturing prediction mistakes due to randomness or unmeasured factors

BRFSS Example

To **predict self-reported general health from BMI**, we seek a function f that captures their relationship. The red curve is our best estimate of it, fitted to the observed data points.



Formal Problem Formulation

Let $\mathcal{X}$ and $\mathcal{Y}$ be two spaces of objects — the sample spaces of two random variables X and Y , with $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ two realisations of them. The goal of predictive modelling is to find a function, or **hypothesis**, h :

$$h : \mathcal{X} \rightarrow \mathcal{Y}$$

which outputs an object $\hat{y} \in \mathcal{Y}$ given an object $x \in \mathcal{X}$. We write y for the **true value** — the ground truth associated with x — and $\hat{y}$ for the **prediction** our function produces:

$$\hat{y} = h(x)$$

Example 1: medical diagnosis

Take $\mathcal{X} = \mathbb{R}^m$ and $\mathcal{Y} = \{0, 1\}$: $\mathbf{x} \in \mathbb{R}^m$ is a numerical vector of blood test results, and $y \in \{0, 1\}$ says whether the subject has the disease (1) or not (0). A suitable h predicts $\hat{y} = h(\mathbf{x})$.

Example 2: a simple spam filter

$\mathcal{X}$ is the set of all possible e-mails and $\mathcal{Y} = \{0, 1\}$. Let $f(x)$ count the spelling mistakes in an e-mail x ; one possible hypothesis is

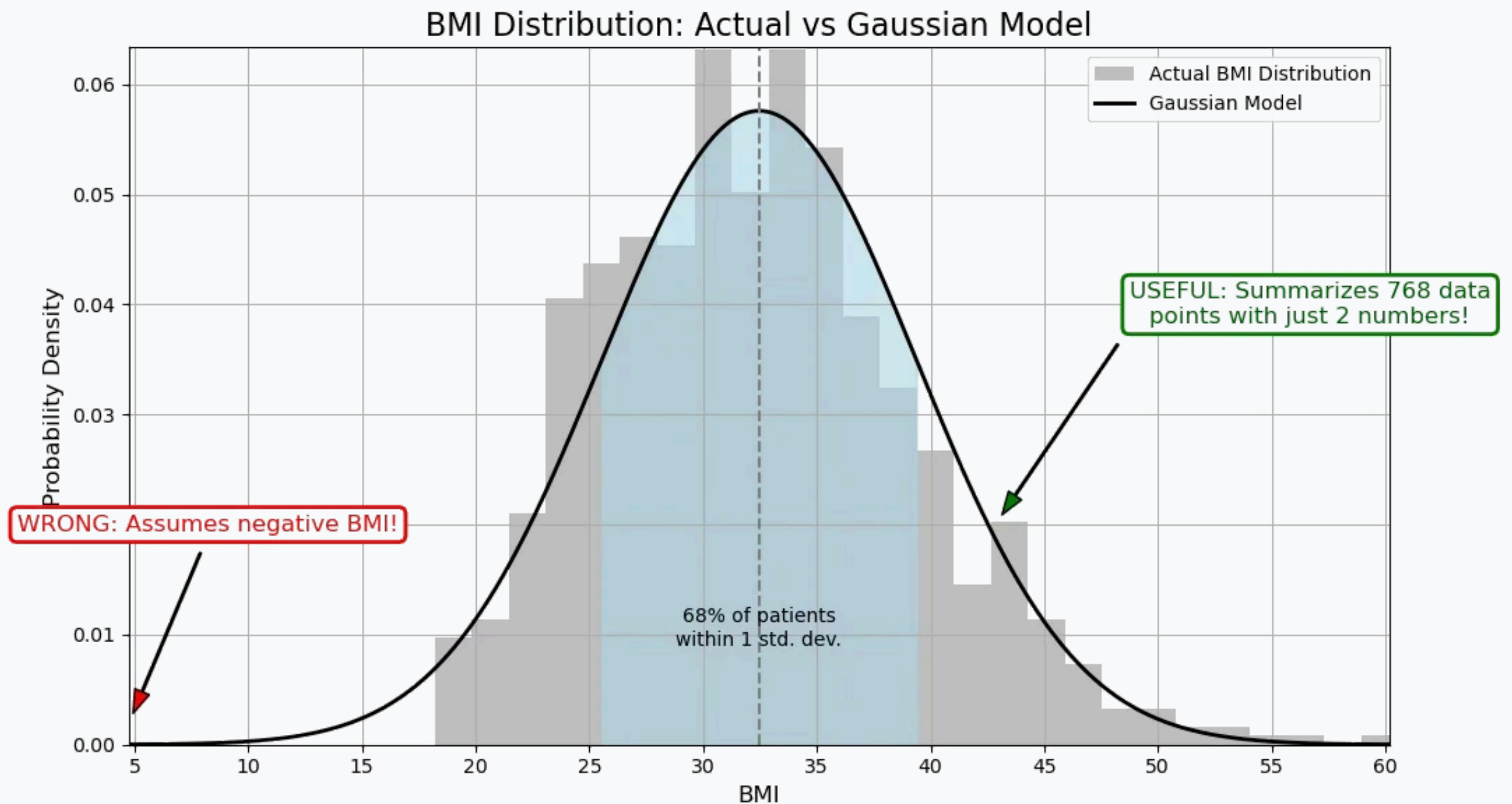
$$h(x) = \begin{cases} 1 & \text{if } f(x) > \theta \\ 0 & \text{otherwise} \end{cases}$$

The second example is worth keeping in mind: it is a legitimate predictive model, it has exactly one parameter, and nothing about it is learned yet. The next slides are about where that θ comes from.



Two Goals: Understanding vs. Predicting

The usefulness of a model depends entirely on our primary objective. In data analysis, we typically pursue two main — sometimes competing — goals that shape every design choice we make.

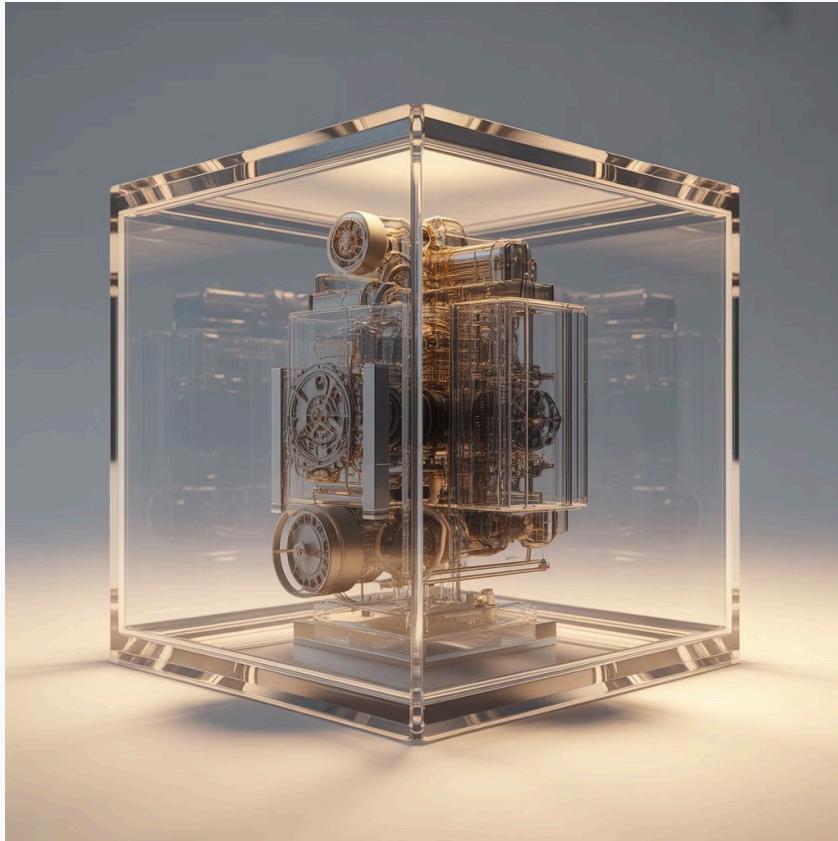


In some cases, the two approaches can overlap in an **integrated approach**.

Example: in diabetes risk management we may seek both to **understand** what the key drivers of diabetes risk are and **predict** which patients are most likely to develop diabetes in the next year.

Statistics vs. Machine Learning

Statistical Approach — the "Glass Box"



Focus: Understanding and Inference

- Relies on assumptions (normality, independence)
- Uses hypothesis testing, p-values, confidence intervals
- Entire dataset used for fitting and testing
- **Interpretability is required**

Key Question: Is this relationship real or random chance?

Machine Learning Approach — the "Black Box"



Focus: Prediction and Accuracy

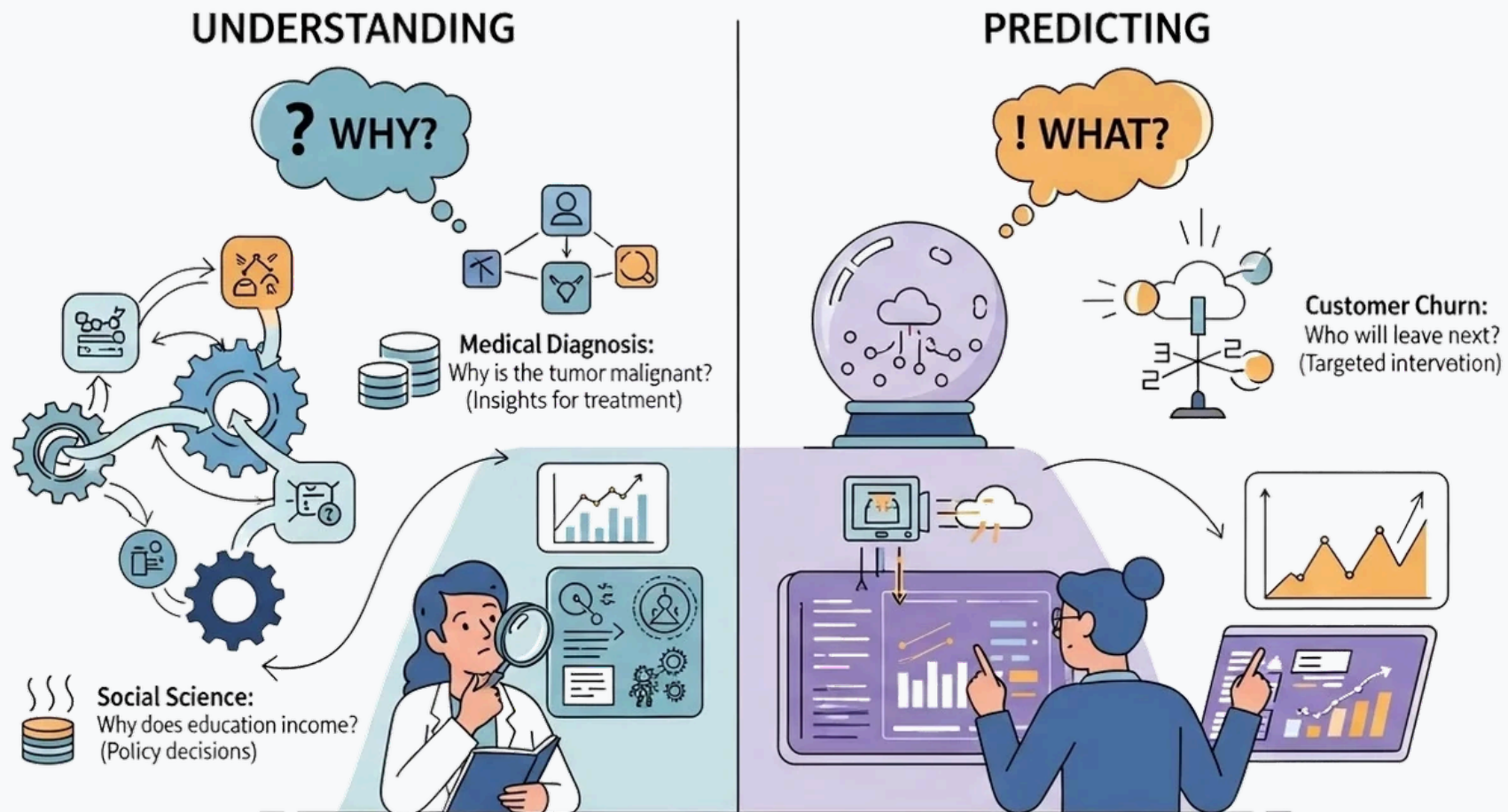
- Results-driven with fewer assumptions
- Splits data into training/validation/test sets
- Success measured by test set performance
- **Interpretability is a bonus**

Key Question: How accurate are predictions on new data?



The Complexity–Interpretability Trade-off

As model complexity increases, interpretability typically decreases. This fundamental tension shapes every modelling decision we make.



Supervised vs. Unsupervised Learning

Supervised Learning — with an Answer Key

Dataset contains both **input features (X)** and **correct answers (y)**.
We "supervise" the model by showing examples with outcomes.

Goal: Learn a mapping function h such that:

$$\hat{y} = h(X)$$

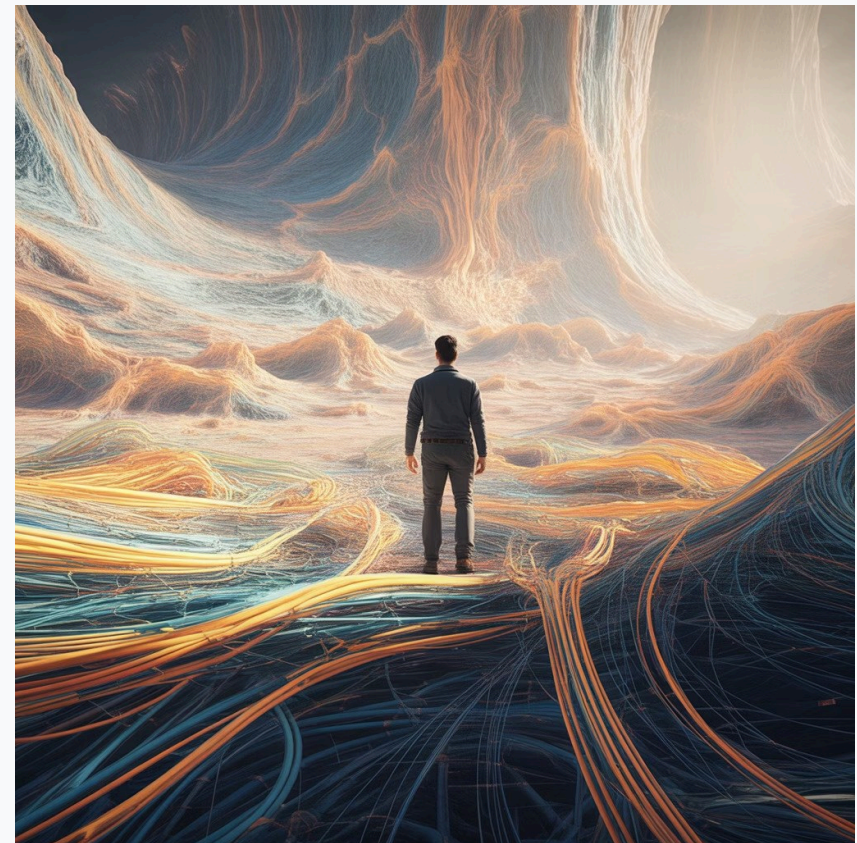


Divided into **Regression** (predicting numbers) and **Classification** (predicting categories).

Unsupervised Learning — without an Answer Key

Dataset contains only **input features (X)** with no correct answers.
Goal is finding **hidden structure or patterns**.

Goal: Discover natural groupings or relationships in data



Primary task is **Clustering** — automatically grouping similar data points together.

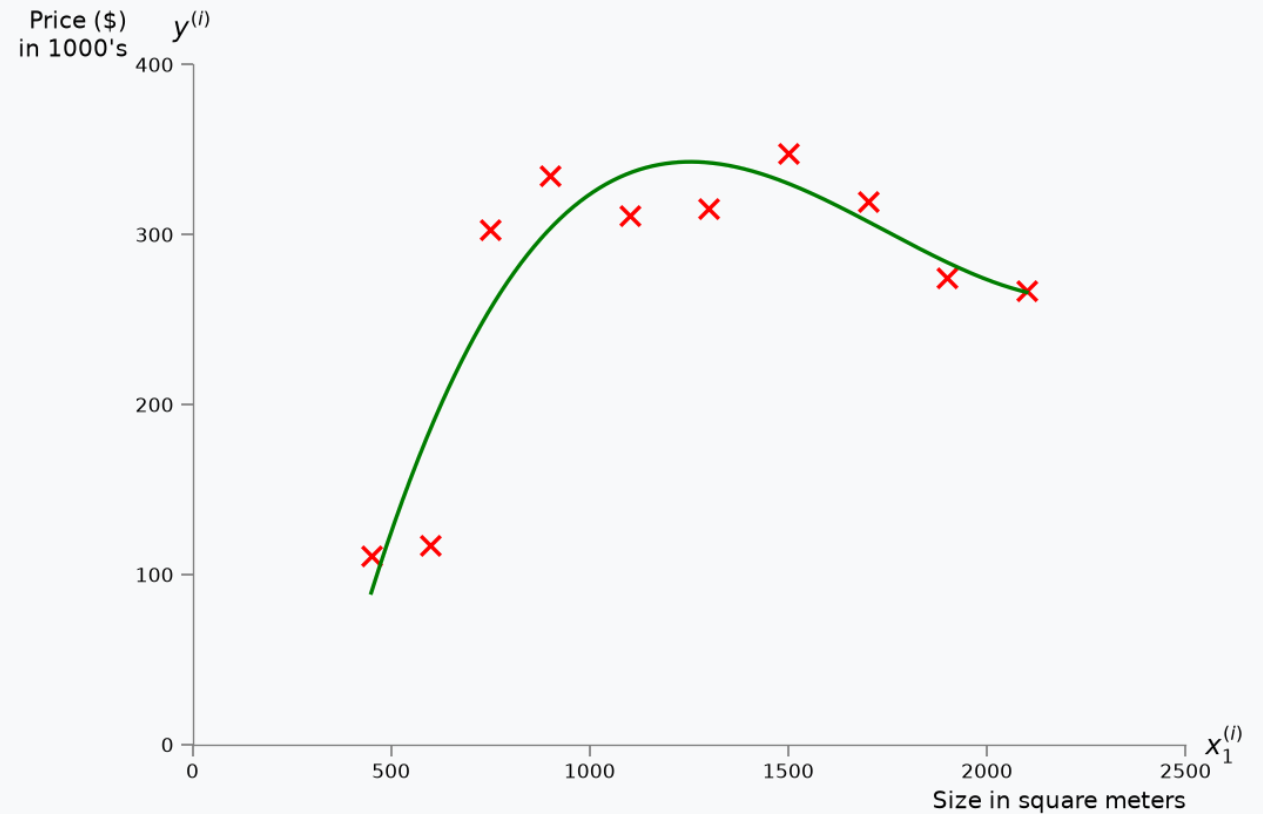
Regression: Predicting Numbers

In regression problems, the output variable y is a **continuous numerical value**. The goal is finding a function (like a line or a curve) that best fits continuous data points.

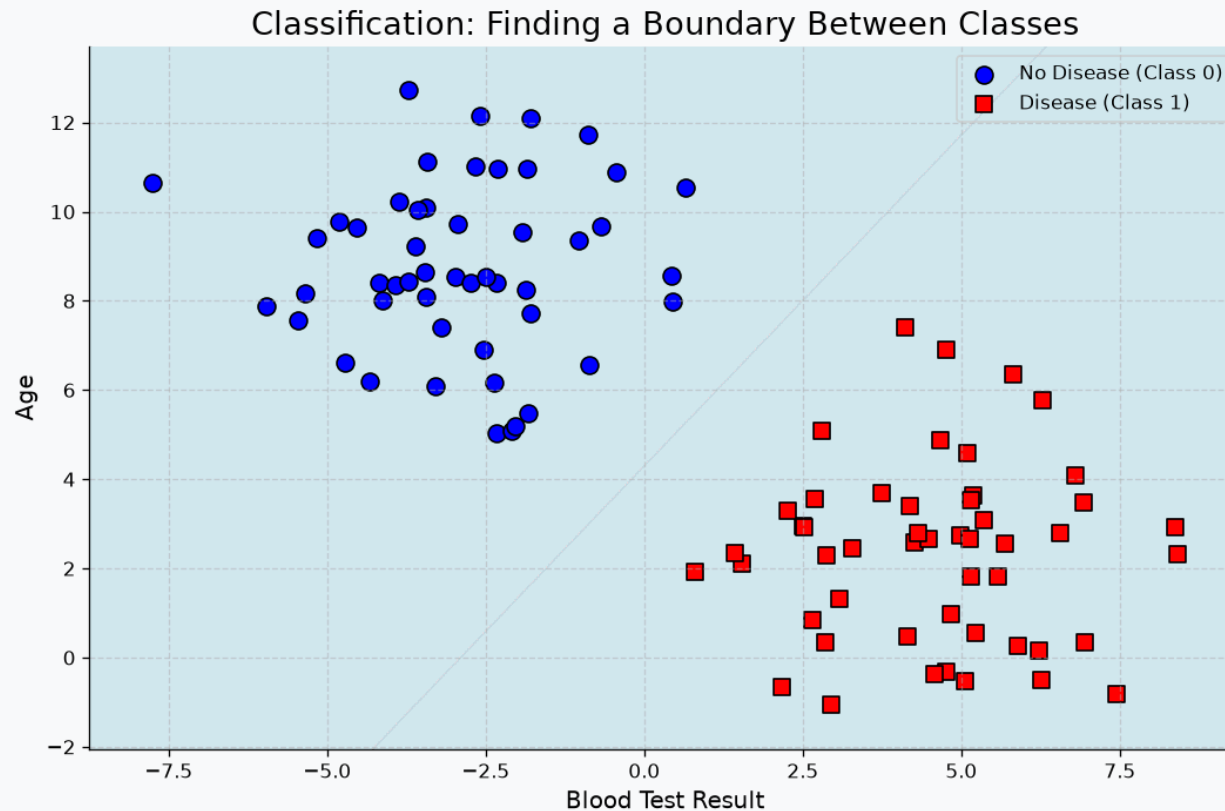
Key Question: "How much?" or "How many?"

Examples include:

- Predicting house prices from square metreage
- Forecasting tomorrow's stock price from historical data
- Estimating a student's exam score from study hours and attendance
- Calculating expected insurance claims based on customer profiles



Classification: Predicting Categories



In classification problems, the output variable y is a **discrete category or label**. The goal is finding a boundary that best separates different labelled groups.

Key Question: "Which one?" or "Is this A or B?"

Examples include:

- **Binary:** Spam detection, medical diagnosis, customer churn prediction
- **Multiclass:** Image recognition (cat, dog, or bird), sentiment analysis (positive, neutral, negative)



Unsupervised Learning: Finding Hidden Structure

In **unsupervised learning**, our dataset contains only input features (X) — no correct answers. The goal isn't predicting an outcome, but discovering hidden structure, patterns, or groupings within the data itself.

Clustering

Goal: Automatically group similar data points into clusters based on their features.

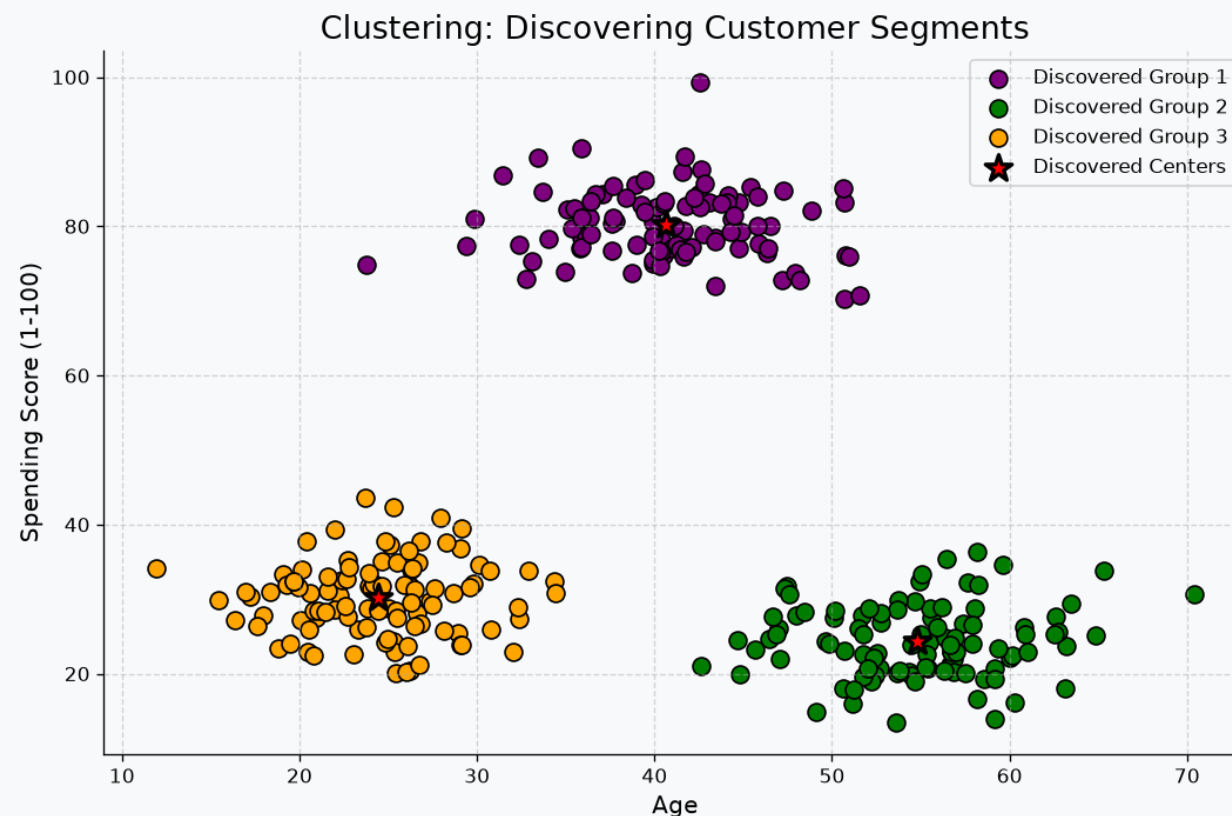
Example: Customer segmentation discovering "Young Savers", "Prime Spenders", and "Older Savers" from age and spending data.

Dimensionality Reduction

Goal: Reduce the number of variables whilst preserving essential information. **Example:** Compressing high-dimensional gene expression data to two dimensions for visualisation.

Anomaly Detection

Goal: Identify unusual patterns that don't conform to expected behaviour. **Example:** Detecting fraudulent transactions by finding patterns that deviate significantly from normal customer behaviour.



Clustering example. Segmenting customers of a bank into three groups based on Age and Spending. Differently from supervised learning, **nobody is giving us the label** and we have to find them ourselves.

Once this segmentation has been performed, we can design targeted marketing campaigns.



Parametric vs. Non-Parametric Models

Parametric Models — the "Fixed" Approach

Makes strong assumptions about function form with a **fixed number of parameters** regardless of dataset size.

Examples: Linear Regression, Logistic Regression, Naive Bayes

- ✓ Simpler and faster to train
- ✓ Requires less data
- ✓ More interpretable
- ✗ Constrained by assumptions
- ✗ Risk of underfitting

This choice embodies the **Bias-Variance Tradeoff**: parametric models are high-bias/low-variance, while non-parametric models are low-bias/high-variance.

Non-Parametric Models — the "Flexible" Approach

Makes few assumptions with **parameters that grow** as training data increases. Lets data "speak for itself."

Examples: K-Nearest Neighbors

- ✓ Highly flexible
- ✓ Captures complex patterns
- ✓ Less bias
- ✗ Requires much more data
- ✗ Slower to train
- ✗ Risk of overfitting



The Learning Principle: Empirical Risk Minimisation

How do we find the "best" model? We need a formal framework.

Let $\mathcal{X}$ and $\mathcal{Y}$ be two spaces of objects. We can see them as the sample spaces of two random variables X and Y , with $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ being two realizations: $X = x$ and $Y = y$.

The goal of predictive modelling is to find a function (or **hypothesis**) h :

$$h : \mathcal{X} \rightarrow \mathcal{Y}$$

which outputs an object $\hat{y} \in \mathcal{Y}$, given an object $x \in \mathcal{X}$. We use the symbol y to refer to the **true value** (or **ground truth**) associated with x , while $\hat{y}$ will represent the prediction obtained using our function h :

$$\hat{y} = h(x)$$

Example 1: Medical Diagnosis

We can imagine $\mathcal{X} = \mathbb{R}^m$ and $\mathcal{Y} = \{0, 1\}$, with $\mathbf{x} \in \mathbb{R}^m$ being a numerical vector representing the results of different blood tests, while $y \in \{0, 1\}$ is a response variable indicating whether the subject has (1) or does not have (0) a given disease.

Example 2: Simple Spam Filter

$\mathcal{X}$ is the set of all possible emails and $\mathcal{Y} = \{0, 1\}$. Let $f(x)$ count the number of orthographical mistakes in an email x . We can define our hypothesis h as:

$$h(x) = \begin{cases} 1 & \text{if } f(x) > \theta \\ 0 & \text{otherwise} \end{cases}$$



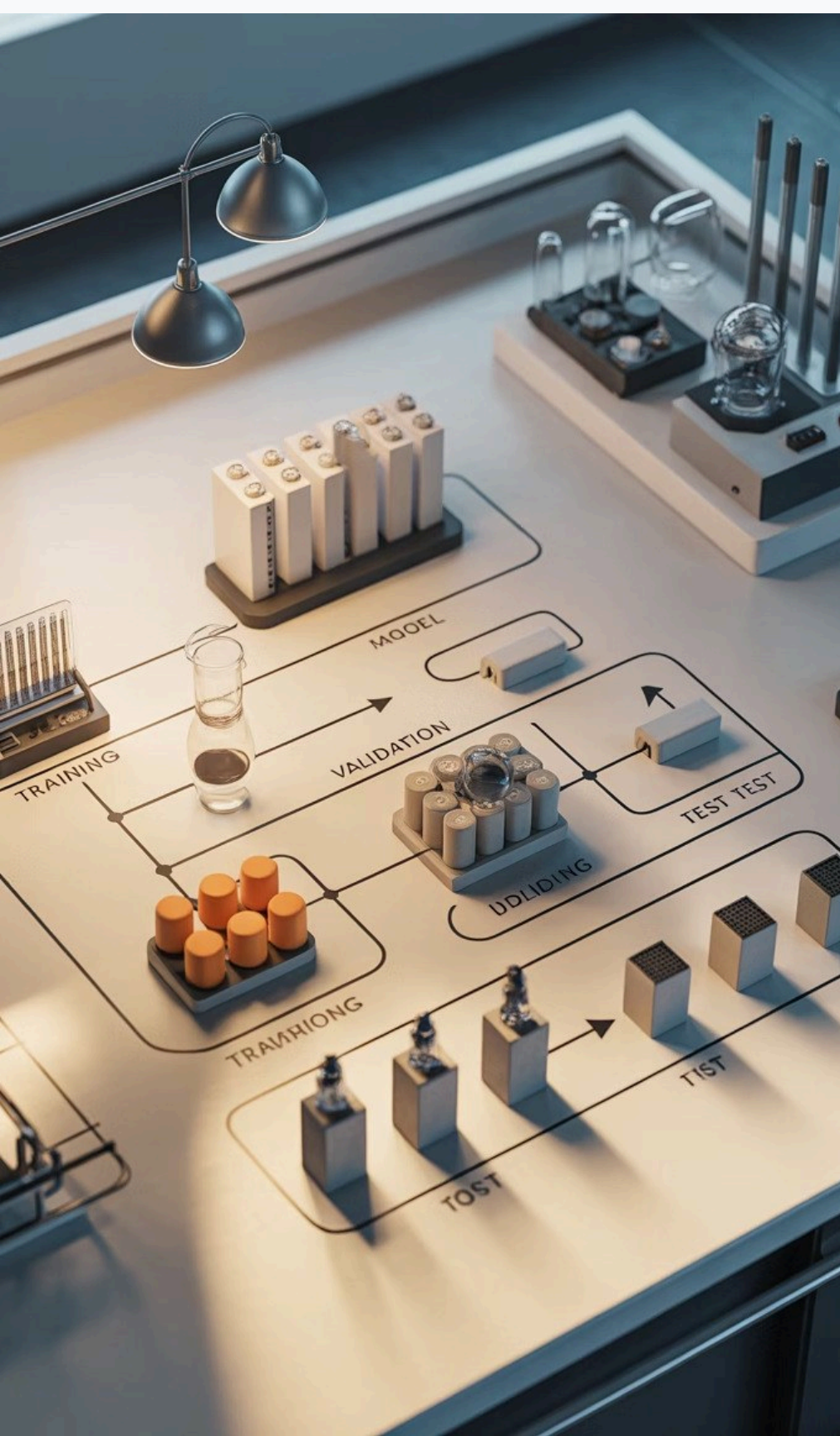
Parameters: What the Model Learns

The function h is typically a **parametric model**, meaning its form is defined by a fixed set of **parameters**. The 'learning' process involves finding the optimal values for these parameters.

For instance, the threshold θ in our spam filter example is a parameter — a 'knob' the algorithm tunes. Each θ value creates a distinct hypothesis function h .

More complex models, like a linear regressor, have multiple parameters such as coefficients $\beta_0, \beta_1, \dots, \beta_n$. The entire set of possible functions h achievable by adjusting these parameters is called the **class of functions (or hypothesis space)** $\mathcal{H}$.

The core question remains: how do we find the **best** parameters?



The Learning Process: Statistical Learning and Empirical Risk

To find the "best" h from our hypothesis space $\mathcal{H}$, we need a way to measure how "good" it is. We do this by defining a **loss (or cost) function**, which measures the penalty for making a prediction $\hat{y}$ when the true value is y :

$$L(\hat{y}, y)$$

Ideally, we want to find the function h^* that has the lowest possible loss *on average over all possible data*, not just the data we have. This "true" average loss is called the **Risk** $R(h)$, defined as the **expected loss** under the true (but unknown) joint probability distribution $P(X, Y)$:

$$R(h) = E_{(x,y) \sim P(X,Y)} [L(h(x), y)]$$

However, we cannot compute true Risk because we don't know the true distribution. Instead, we estimate it using our training set of N examples, $TR = \{(x_i, y_i)\}_{i=1}^N$. This leads to **Empirical Risk**:

$$R_{emp}(h) = \frac{1}{N} \sum_{i=1}^N L(h(x_i), y_i)$$

Once we have this expression, we find our optimal $\hat{h}$ by solving this optimisation problem:

$$\hat{h} = \arg \min_{h \in \mathcal{H}} R_{emp}(h)$$

01

Define Loss Function

Choose how to penalise prediction errors (e.g., squared error for regression)

02

Calculate Empirical Risk

Average the loss over all training examples

03

Minimise Risk

Find parameters that minimise empirical risk using optimisation algorithms



Model Capacity: Flexibility vs. Complexity

Before moving on, we need to define the crucial concept of model capacity. In simple terms, a model's capacity refers to its **flexibility** or **complexity**, directly related to the **size and richness of its hypothesis space $\mathcal{H}$** .

Low Capacity: Simple & Constrained

Has a **small, constrained** hypothesis space $\mathcal{H}$, representing a limited set of simple functions.

Example: A simple linear regression model ($h(x) = \beta_0 + \beta_1 x$) can *only* learn linear patterns. It is "biased" towards simplicity.

Risk: May be too simple to capture the true underlying data patterns, leading to **underfitting**.

High Capacity: Flexible & Complex

Possesses a **large, rich** hypothesis space $\mathcal{H}$, capable of representing complex functions.

Example: A 10th-degree polynomial can fit highly "wiggly" functions, perfectly matching almost any training points.

Risk: So flexible it can fit random *noise* in training data, not just the signal. Leads to **overfitting** and poor generalisation.



Assessing Model Accuracy

Once we use Empirical Risk Minimisation to "learn" a model $\hat{h}$, we must evaluate its effectiveness. A model that perfectly fits our training data (i.e., has a low R_{emp}) is **not necessarily a good model**. Our ultimate goal is a model that **generalises** well — one that makes accurate predictions on new, unseen data.

Measuring the Quality of Fit

To measure the performance of a model, we need a concrete **evaluation metric**. This metric is sometimes the same as the loss function $L(\hat{y}, y)$ we used in the ERM principle. The specific metric we choose depends on the problem.

For **regression problems**, the most common metric is the **Mean Squared Error (MSE)**. If we choose our loss function $L(y, \hat{y}) = (y - \hat{y})^2$, then the MSE is precisely the Empirical Risk we are trying to minimise:

$$R_{emp}(h) = \frac{1}{N} \sum_{i=1}^N (y_i - h(x_i))^2 = \text{MSE}$$

This leads to a critical question: is a model with a very low MSE on our *training data* a good model? **Not necessarily.**



Overfitting and Underfitting

The central challenge of predictive modelling: a model that perfectly fits training data isn't necessarily good. Models can be too simple (**underfitting**) or too complex (**overfitting**).

For example, this model:

$$\hat{h}(x) = y \text{ s.t. } (x, y) \in TR$$

Leads to an empirical risk equal to zero, but the function is not even defined for values outside the training set — an extreme case of overfitting where the model is just "memorising the data".

Underfitting

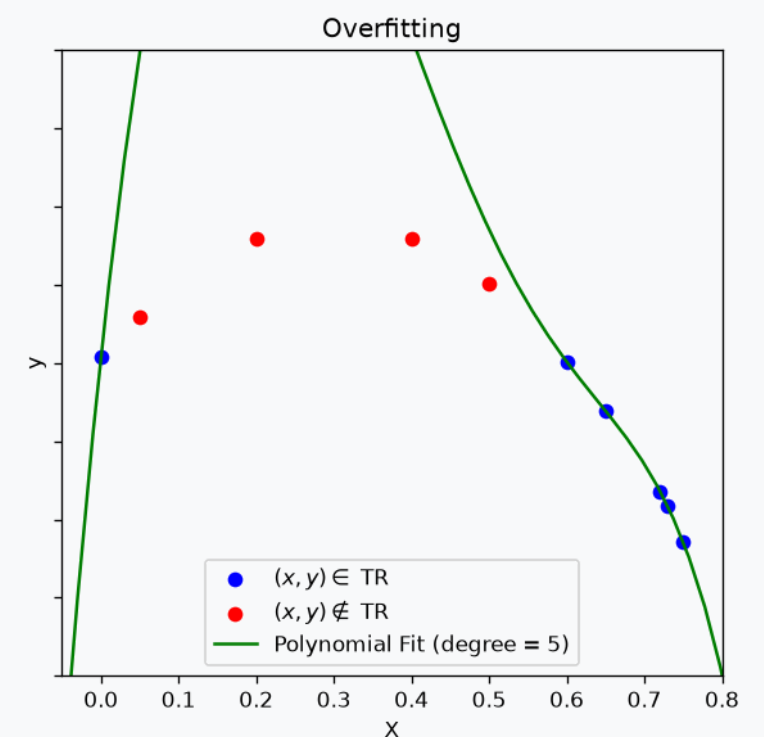
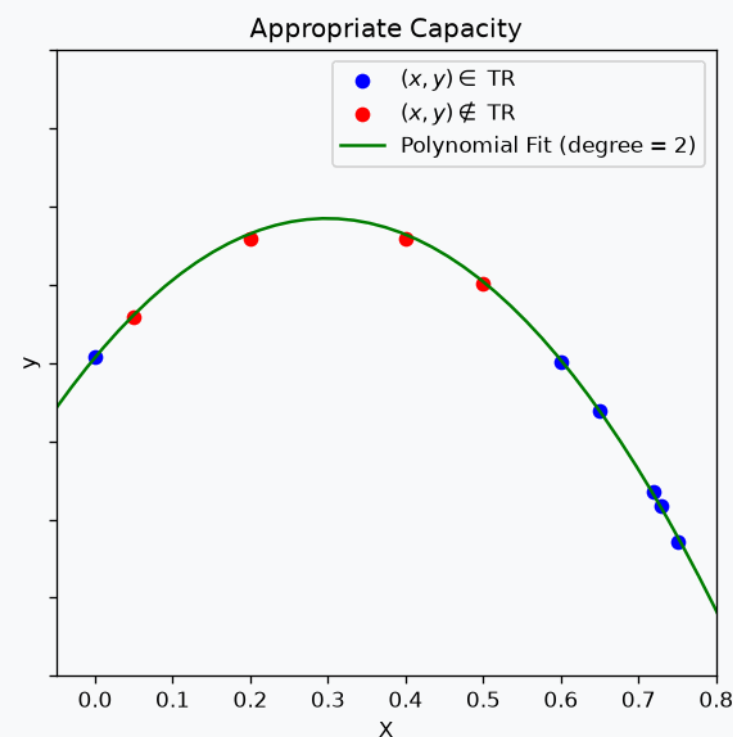
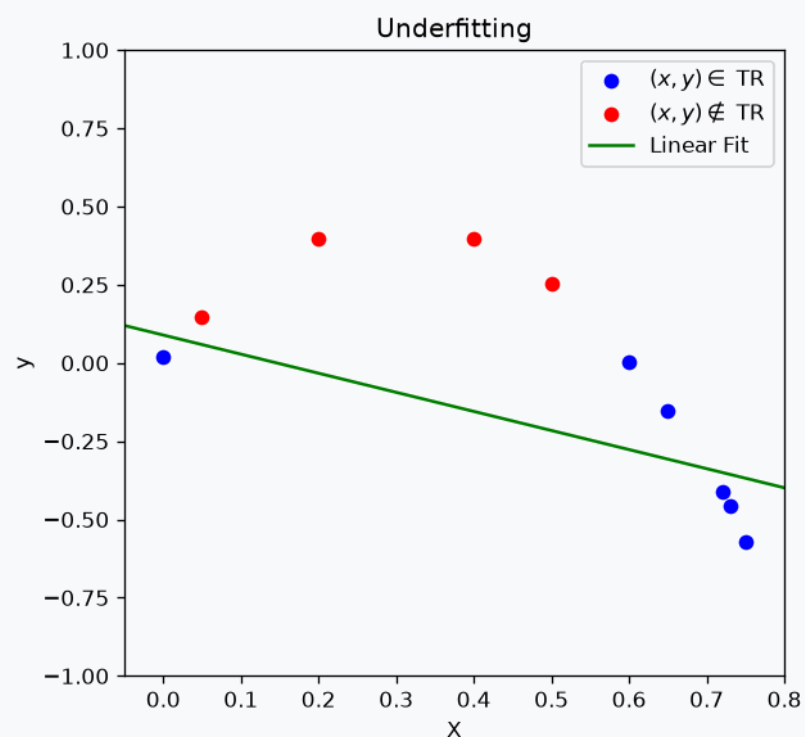
Model is too simple to capture underlying patterns. High training error, high test error. Solution: increase model capacity.

Good Fit

Model captures signal without memorising noise. Moderate training error, low test error. Optimal bias-variance trade-off.

Overfitting

Model memorises training data including noise. Very low training error, high test error. Solution: reduce model capacity or add regularisation.



The Bias-Variance Trade-off

The expected test error at a given point x_0 decomposes into three fundamental quantities:

$$E[(y_0 - \hat{f}(x_0))^2] = [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\hat{f}(x_0)) + \text{Var}(\epsilon)$$

Squared Bias

Error from underfitting. High-bias models are "stubborn" — stuck in simple assumptions. They miss the true pattern, producing high error on both training and test sets.

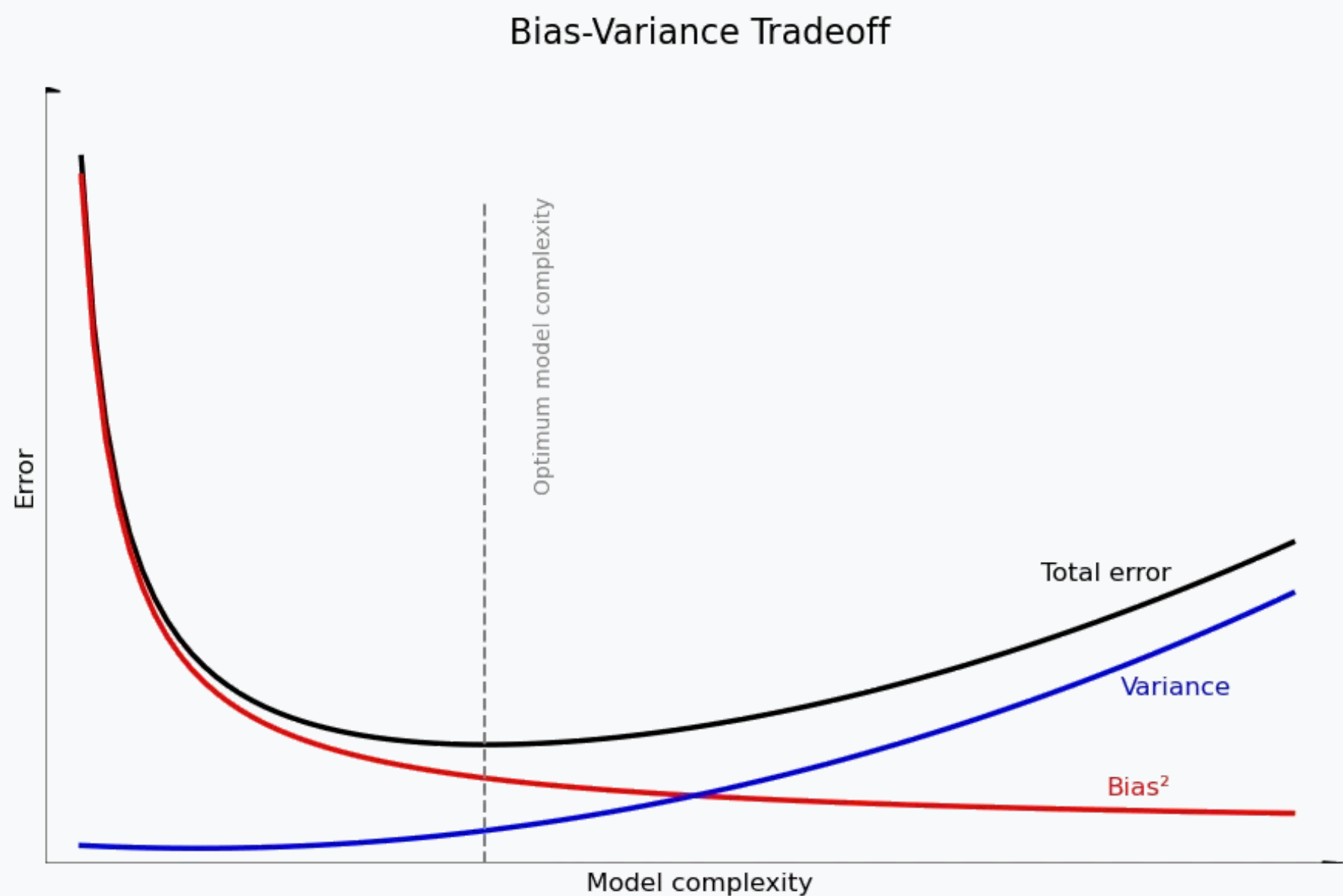
Variance

Error from overfitting. High-variance models are "nervous" — too sensitive to training data specifics. They fit training noise perfectly but fail on test data.

Irreducible Error

Inherent noise in real-world data that no model can eliminate. Represents the upper limit on achievable accuracy.

As model capacity increases, bias decreases (good) but variance increases (bad). Our goal: find the optimal capacity that minimises total error.



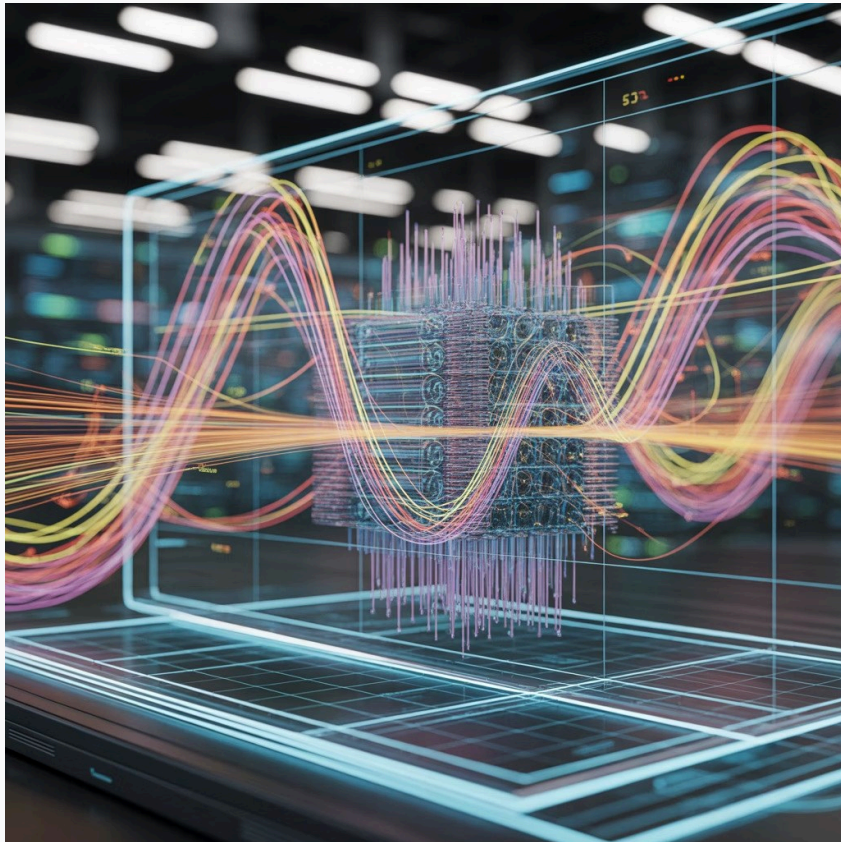
Parameters vs. Hyperparameters

Model Parameters

Values the model **learns from data** during training through Empirical Risk Minimization. Internal to the model, direct output of the learning algorithm.

Examples:

- Coefficients $\beta_0, \beta_1, \dots$ in linear regression
- Weights in neural networks
- Probabilities in Naive Bayes



Hyperparameters

Configuration settings set **before training begins**. External to model, control capacity and the learning process — the "knobs" we tune.

Examples:

- Polynomial degree k (controls capacity)
- Regularization parameter λ
- Number of clusters k in K-Means



- **Critical Challenge:** Learning algorithms find the best model parameters for *given* hyperparameters, but cannot choose optimal hyperparameters. That's our job — how we find the bias-variance sweet spot.

Model Selection Strategies

Statistical Selection (Understanding)

For **inference goals**, select models based on explaining observed data, favouring simplicity. **Uses the entire dataset** with statistical measures:

- p-values for variable significance
- Adjusted R^2 balancing fit and complexity

Not trying to predict the future, but to find a trustworthy explanation.

Predictive Selection (Predicting)

For **prediction goals**, the only criterion is performance on unseen data. **Splits data** to simulate the future:

- Training set: optimise the model
- Validation set: choose hyperparameters
- Test set: final performance evaluation

Common ratio: 60% train, 20% validation, 20% test.

Golden Rule: Performance on training data is irrelevant. Only performance on unseen data matters.



Performance Measures

When our main goal is **prediction**, our selection criteria change entirely. We no longer care about statistical significance; we focus solely on performance on unseen data.

The Golden Rule of Predictive Modelling: The performance of a model on the data it was trained on is *irrelevant*. The *only* measure that matters is its performance on new, unseen data.

To adhere to this rule, we simulate future performance by splitting our existing data into distinct sets for training, validation, and testing.

Loss Function (for Training)

Used internally by the learning algorithm to guide model optimisation. Often chosen for its mathematical properties, even if it's an approximation of our true objective.

Performance Measure (for Evaluation)

Our true objective function to quantify a model's quality. Typically 'the higher the better' (e.g., accuracy) or 'the lower the better' (e.g., error rate).

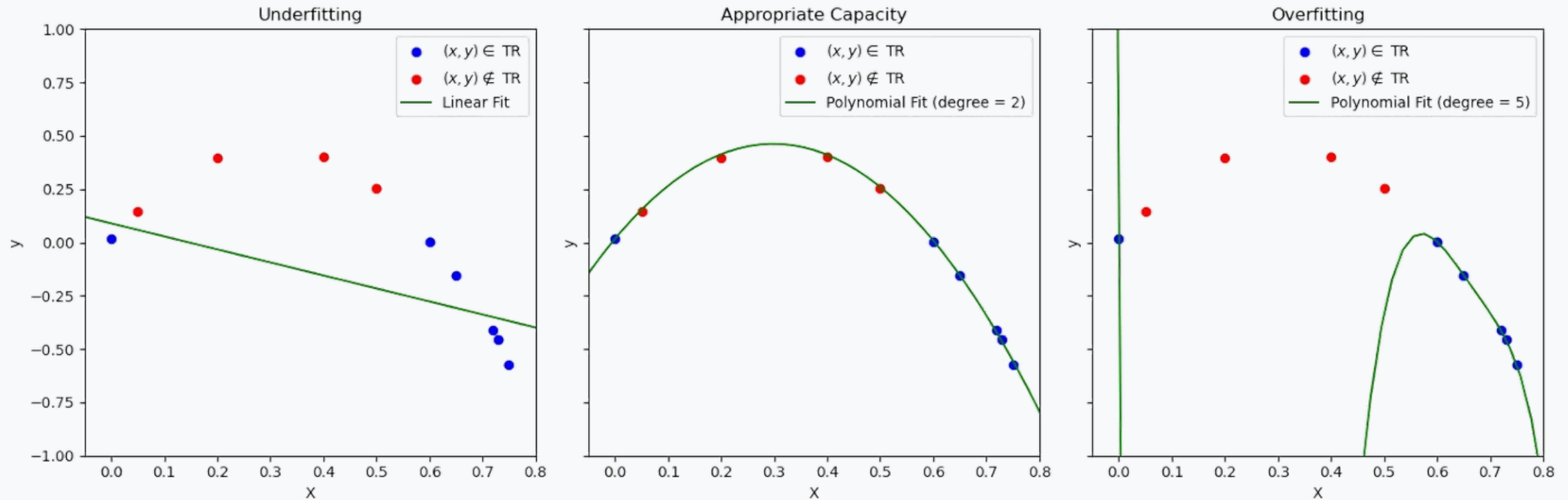
A performance measure p evaluates the set of ground truth values Y against the set of corresponding predictions $\hat{Y}$ for a given dataset:

$$p : \mathcal{Y}^N \times \mathcal{Y}^N \rightarrow \mathbb{R}$$



Holdout Validation, or Single Split

Split the data into two parts: training and test set following some ratio, e.g., using 70 — 80% of the data for training and the rest for testing.



K-Fold Cross-Validation

If the dataset is small, divide it into K folds, then use $K - 1$ folds for training and the rest for validation. Aggregate results across folds (e.g., with the mean).



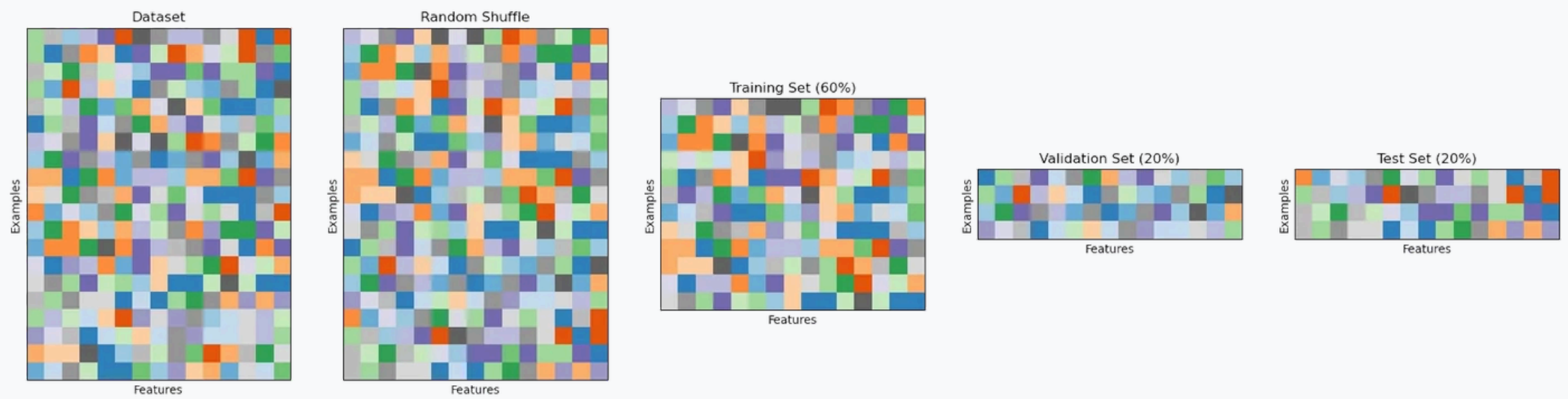
Leave-One-Out Cross-Validation

When the dataset is very small, we come to an extreme of K-Fold validation where **each fold contains a single element**.

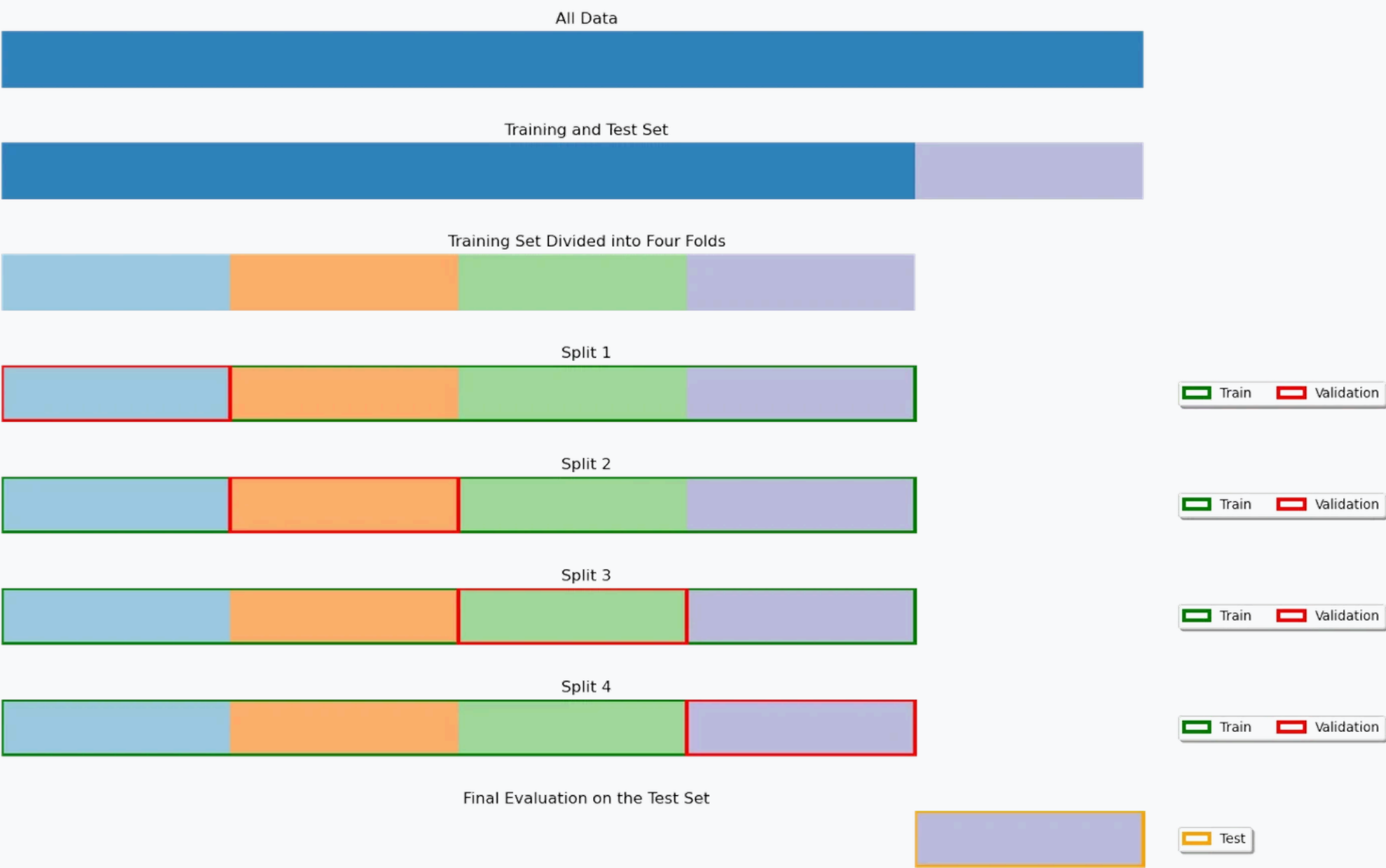


Model Selection and Hyperparameter Optimisation

When we need to tune hyper-parameters, we need three sets: a **training set** (for training), a **validation set** (to choose hyperparameters), and a **test set** (to test final performance). Often we split in a 60 : 20 : 20 ratio, but other ratios are also common.



Other combinations are also possible. For instance holdout validation plus cross-validation only for hyperparameters:



Conclusions and Next Steps



We Have Explored:

The fundamentals of predictive analysis

Prediction against inference

Statistics against machine learning

The problem landscape

Parametric and non-parametric models

Empirical risk minimisation

Model accuracy and model selection

- In the next lecture we put this to work on regression: overfitting, regularisation and the scikit-learn workflow.

References

- Chapter 2 of James, Gareth Gareth Michael. *An introduction to statistical learning: with applications in Python*, 2023.
<https://www.statlearning.com>
- Empirical risk minimization · scikit-learn: cross-validation
- Lecture notes: **Chapter 13 — Introduction to predictive analysis**

